МИНСПОРТ РОССИИ

ФСП федерация спортивного программирования России

КУБОК ФЕДЕРАЦИИ 2026

РАЙТАП · REVERSE

АУДИТ

REVERSE

19 СЕНТЯБРЯ 2026
ОНЛАЙН · CTF TASK-BASED

REVERSE
КАТЕГОРИЯ

22
СЛАЙДОВ В РАЗБОРЕ

01

ЧТО ДАЛИ

В задании лежит один 64-битный Windows-бинарь `WorkstationAudit.exe`. По легенде это корпоративный чекер: он проверяет настройку рабочей станции и при успешном прохождении всех пунктов печатает отчетный токен.

Сразу фиксирую файл, с которым работаю:

```
WorkstationAudit.exe
sha256: af5af7a1c6b12dcc72eaa04f07f1edc907955ecad7afbd9f896d69e6ef7a4117
format: PE32+ executable, x86-64, console
```

Запускать неизвестный PE на основной машине не требуется. Весь разбор ниже статический.

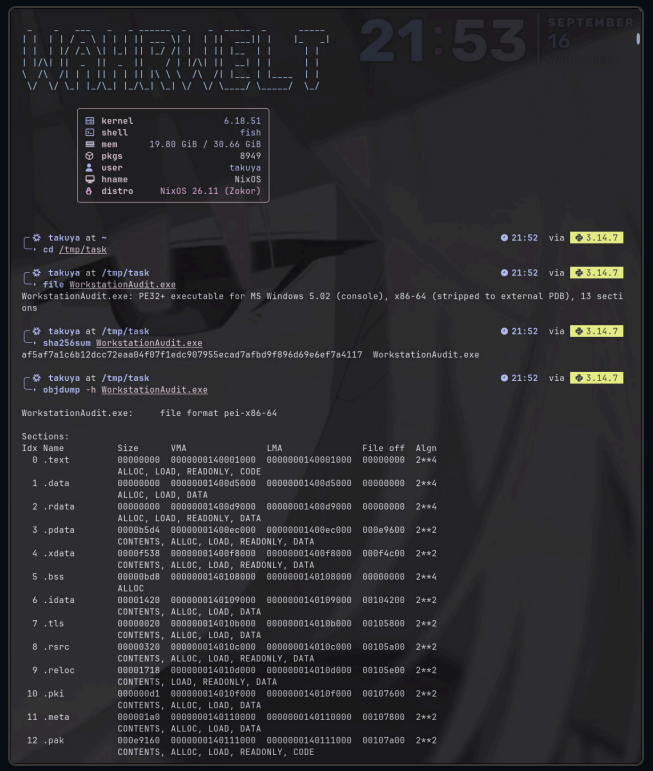
АУДИТ
REVERSE

02

ПЕРВИЧНЫЙ ОСМОТР

Начинаю с обычных `file`, `strings` и таблицы секций:

```
file files/WorkstationAudit.exe
sha256sum files/WorkstationAudit.exe
objdump -h files/WorkstationAudit.exe
strings -a -n 5 files/WorkstationAudit.exe | less
```



АУДИТ
REVERSE

02

ПЕРВИЧНЫЙ ОСМОТР

ПРОДОЛЖЕНИЕ

У файла странная раскладка:

```
.text   virtual 0x000d4000, raw 0
.data   virtual 0x00004000, raw 0
.rdata  virtual 0x00013000, raw 0
.meta   raw 0x000001a0
.pak    raw 0x000e9160
```

Точка входа находится по RVA `0x111000`, то есть прямо в `.pak`. Нормальные `.text`, `.data` и `.rdata` занимают место в памяти, но их содержимого в обычных raw-секциях нет. В строках исходного файла остаются в основном импорты маленького загрузчика и случайный шум.

Значит передо мной не основной код чекера, а упаковщик. Его задача - восстановить исходные секции в памяти и передать управление настоящему entry point.

АУДИТ
REVERSE

03

РАЗБИРАЕМ ЗАГРУЗЧИК

Открываю файл в IDA и начинаю с `start` по адресу `0x140111000`. В начале функция расшифровывает 336 байт из `.meta`. Используется обычный 32-битный LCG:

```
state = state * 1664525 + 1013904223 mod 2^32
byte ^= state >> 24
```

Начальное состояние равно `2050694495`. Байты метаданных `0x40-0x43` загрузчик специально пропускает.



```
__int64 start()
{
  __int64 v0; // rax
  int v1; // edx
  __int64 v2; // rsi
  int v3; // r15d
  __int64 (__fastcall *v4)(__int64); // r13
  __int64 (__fastcall *v5)(__int64, __int64); // r12
  __int64 v6; // rax
  void (__fastcall *v7)(__int64, __int64, __int64, _DWORD *); // r14
  __int64 v8; // rdx
  __int64 v9; // rdi
  __int64 v10; // rbp
  __int64 v11; // rcx
  __int64 v12; // rax
  int v13; // edi
  unsigned int *i; // rdi
  __int64 v15; // r15
  __int64 v16; // rcx
  __int64 v17; // rbp
  __int64 v18; // rax
  __int64 *v19; // r14
  __int64 v20; // rbp
  __int64 v21; // rdx
  _QWORD *v22; // r15
  int v24; // [rsp+20h] [rbp-78h]
  __int64 v25; // [rsp+20h] [rbp-78h]
  __int64 v26; // [rsp+30h] [rbp-68h]
  unsigned int v27; // [rsp+38h] [rbp-60h]
  unsigned int v28; // [rsp+3Ch] [rbp-5Ch]
  _DWORD v29[19]; // [rsp+4Ch] [rbp-4Ch] BYREF

  v0 = 0;
  v1 = 2050694495;
  do
  {
    while ( 1 )
    {
      v1 = 1664525 * v1 + 1013904223;
      if ( (unsigned int)(v0 - 64) > 3 )
        break;
      ++v0;
    }
    *(_BYTE *)(v0 + 0x140110000LL) ^= HIBYTE(v1);
    ++v0;
  }
  while ( v0 != 336 );
  v2 = unk_140110000;
  v3 = 0;
```

03

РАЗБИРАЕМ ЗАГРУЗЧИК

ПРОДОЛЖЕНИЕ

После расшифровки структура `.meta` становится читаемой:

```
image base       0x140000000
payload source   0x1401111d0
original OEP RVA 0x1480
import RVA       0x109000
global seed      0x9e3779b9
chunk count      3
```

Сразу после заголовка лежат три дескриптора по 16 байт:

TARGET RVA	SIZE	OFFSET В PAYLOAD	PROTECTION
`0x1000`	`0xd3340`	`0x0`	`0x20`
`0xd5000`	`0x3170`	`0xd3340`	`0x4`
`0xd9000`	`0x12ae0`	`0xd64b0`	`0x2`

Это ровно диапазоны с кодом, изменяемыми данными и строками. Для каждого дескриптора упаковщик вызывает `VirtualProtect`, берет начальное состояние `0x9e3779b9 + target_rva` и расшифровывает payload тем же LCG.

АУДИТ
REVERSE

03

РАЗБИРАЕМ ЗАГРУЗЧИК

ПРОДОЛЖЕНИЕ

```
while ( v3 != v24 )
{
  v8 = dword_140110050[4 * v3 + 1];
  v9 = dword_140110050[4 * v3];
  v28 = dword_140110050[4 * v3 + 1];
  v10 = unk_140110018 + dword_140110050[4 * v3 + 2];
  v27 = dword_140110050[4 * v3 + 3];
  v26 = v8;
  v29[0] = 0;
  v7(a1: v2 + v9, a2: v8, a3: 64, a4: v29);
  v11 = v2 + v9;
  v12 = 0;
  v13 = unk_140110044 + v9;
  while ( (unsigned int)v12 < v28 )
  {
    v13 = 1664525 * v13 + 1013904223;
    *(_BYTE *)(v11 + v12) = *(_BYTE *)(v10 + v12) ^ HIBYTE(v13);
    ++v12;
  }
  ++v3;
  v7(a1: v11, a2: v26, a3: v27, a4: v29);
}
if ( dword_14011003C != 0 )
  *(_DWORD *)(dword_14011003C + v2) = TlsIndex;
for ( i = (unsigned int *)(v2 + unk_140110034); ; i += 5 )
{
  v15 = *i;
  v16 = i[3];
  v17 = i[4];
  if ( ((unsigned int)v17 | (unsigned int)v16 | *i) == 0 )
    break;
  v18 = v4(a1: v2 + v16);
  if ( (_DWORD)v15 == 0 )
    v15 = (unsigned int)v17;
  v25 = v18;
  v19 = (__int64 *)(v15 + v2);
  v20 = v17 - v15;
  while ( 1 )
  {
    v21 = *v19;
    if ( *v19 == 0 )
      break;
    v22 = (__int64 *)((char *)v19 + v20);
    if ( *v19 >= 0 )
      v21 = v2 + (unsigned int)v21 + 2;
    else
      v21 = (unsigned __int16)v21;
    ++v19;
    *v22 = v5(a1: v25, a2: v21);
00107AEF start:70 (1401110EF)
```

АУДИТ
REVERSE

03

РАЗБИРАЕМ ЗАГРУЗЧИК

ПРОДОЛЖЕНИЕ

После этого загрузчик восстанавливает импорты, вызывает TLS callback и переходит на исходный entry point `0x140001480`.

АУДИТ
REVERSE

04

ВОССТАНАВЛИВАЕМ MEMORY IMAGE

Чтобы не копировать байты вручную из отладчика, я написал unpack.py. Скрипт:

1. Разбирает PE-заголовок и отображает raw-секции по их RVA.
2. Расшифровывает `.meta`.
3. Читает из метаданных адрес payload, seed и таблицу диапазонов.
4. Восстанавливает три исходных диапазона в memory image.

Запуск дает ожидаемую карту:

```
python3 unpack.py
```

```
image_base=0x140000000
entry=0x140001480
chunk rva=0x1000 size=0xd3340 source=0x0 protection=0x20
chunk rva=0xd5000 size=0x3170 source=0xd3340 protection=0x4
chunk rva=0xd9000 size=0x12ae0 source=0xd64b0 protection=0x2
wrote files/WorkstationAudit.mem (2076672 bytes)
```

Получившийся `WorkstationAudit.mem` - это memory image для анализа, а не файл для запуска. Исходный exe скрипт не меняет.

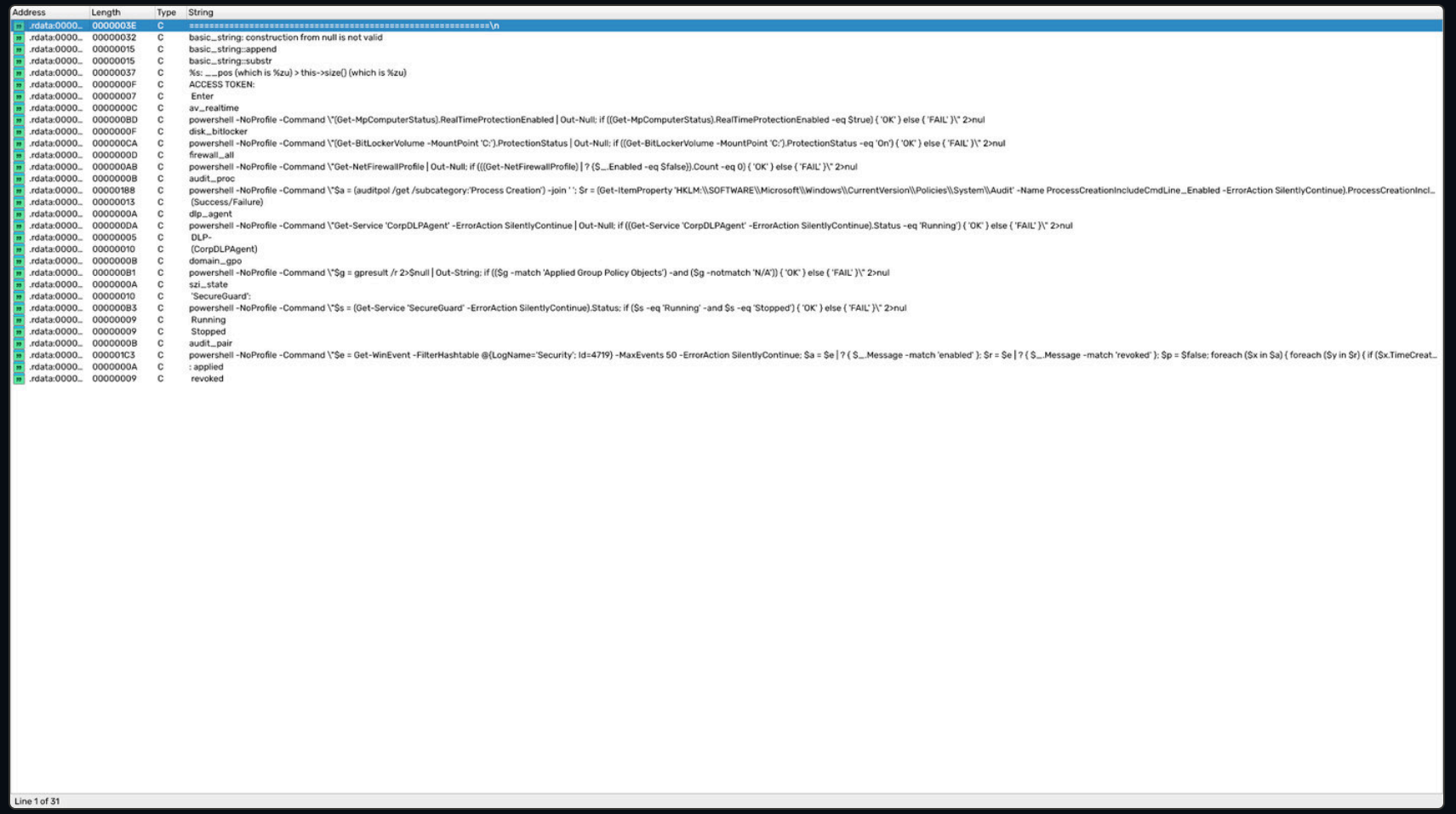
После загрузки восстановленных диапазонов в IDA появляются строки настоящего чекера. Среди них сразу видны `ACCESS TOKEN`, названия восьми проверок и PowerShell-команды.

АУДИТ
REVERSE

04

ВОССТАНАВЛИВАЕМ MEMORY IMAGE

ПРОДОЛЖЕНИЕ



Address	Length	Type	String
.rdata:0000…	0000003E	C	==========\n
.rdata:0000…	00000032	C	basic_string: construction from null is not valid
.rdata:0000…	00000015	C	basic_string::append
.rdata:0000…	00000015	C	basic_string::substr
.rdata:0000…	00000037	C	%s: __pos (which is %zu) > this->size() (which is %zu)
.rdata:0000…	0000000F	C	ACCESS TOKEN:
.rdata:0000…	00000007	C	Enter
.rdata:0000…	0000000C	C	av_realtime
.rdata:0000…	000000BD	C	powershell -NoProfile -Command \"(Get-MpComputerStatus).RealTimeProtectionEnabled | Out-Null; if ((Get-MpComputerStatus).RealTimeProtectionEnabled -eq $true) { 'OK' } else { 'FAIL' }\" 2>nul
.rdata:0000…	0000000F	C	disk_bitlocker
.rdata:0000…	000000CA	C	powershell -NoProfile -Command \"(Get-BitLockerVolume -MountPoint 'C:').ProtectionStatus | Out-Null; if ((Get-BitLockerVolume -MountPoint 'C:').ProtectionStatus -eq 'On') { 'OK' } else { 'FAIL' }\" 2>nul
.rdata:0000…	0000000D	C	firewall_all
.rdata:0000…	000000AB	C	powershell -NoProfile -Command \"Get-NetFirewallProfile | Out-Null; if (((Get-NetFirewallProfile) | ? {$_.Enabled -eq $false}).Count -eq 0) { 'OK' } else { 'FAIL' }\" 2>nul
.rdata:0000…	0000000B	C	audit_proc
.rdata:0000…	000001B8	C	powershell -NoProfile -Command \"$a = (auditpol /get /subcategory:'Process Creation') -join ' '; $r = (Get-ItemProperty 'HKLM:\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Policies\\System\\Audit' -Name ProcessCreationIncludeCmdLine_Enabled -ErrorAction SilentlyContinue).ProcessCreationIncl…
.rdata:0000…	00000013	C	(Success/Failure)
.rdata:0000…	0000000A	C	dlp_agent
.rdata:0000…	000000DA	C	powershell -NoProfile -Command \"Get-Service 'CorpDLPAgent' -ErrorAction SilentlyContinue | Out-Null; if ((Get-Service 'CorpDLPAgent' -ErrorAction SilentlyContinue).Status -eq 'Running') { 'OK' } else { 'FAIL' }\" 2>nul
.rdata:0000…	00000005	C	DLP-
.rdata:0000…	00000010	C	(CorpDLPAgent)
.rdata:0000…	0000000B	C	domain_gpo
.rdata:0000…	000000B1	C	powershell -NoProfile -Command \"$g = gpresult /r 2>$null | Out-String; if (($g -match 'Applied Group Policy Objects') -and ($g -notmatch 'N/A')) { 'OK' } else { 'FAIL' }\" 2>nul
.rdata:0000…	0000000A	C	szi_state
.rdata:0000…	00000010	C	'SecureGuard':
.rdata:0000…	000000B3	C	powershell -NoProfile -Command \"$s = (Get-Service 'SecureGuard' -ErrorAction SilentlyContinue).Status; if ($s -eq 'Running' -and $s -eq 'Stopped') { 'OK' } else { 'FAIL' }\" 2>nul
.rdata:0000…	00000009	C	Running
.rdata:0000…	00000009	C	Stopped
.rdata:0000…	0000000B	C	audit_pair
.rdata:0000…	000001C3	C	powershell -NoProfile -Command \"$e = Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4719} -MaxEvents 50 -ErrorAction SilentlyContinue; $a = $e | ? { $_.Message -match 'enabled' }; $r = $e | ? { $_.Message -match 'revoked' }; $p = $false; foreach ($x in $a) { foreach ($y in $r) { if ($x.TimeCreat…
.rdata:0000…	0000000A	C	: applied
.rdata:0000…	00000009	C	revoked

Line 1 of 31

АУДИТ
REVERSE

04

ВОССТАНАВЛИВАЕМ MEMORY IMAGE

ПРОДОЛЖЕНИЕ

Одна проверка особенно хорошо показывает, что честно получить токен на обычной машине не предполагается:

```
$s = (Get-Service 'SecureGuard').Status
if ($s -eq 'Running' -and $s -eq 'Stopped') { 'OK' } else { 'FAIL' }
```

Одна и та же служба должна одновременно иметь состояния `Running` и `Stopped`. Значит задача действительно про восстановление генератора токена, а не про настройку Windows.

АУДИТ
REVERSE

05

НАХОДИМ МЕСТО ПЕЧАТИ ТОКЕНА

Строка `ACCESS TOKEN:` находится по адресу `0x1400d9275`. RIP-relative ссылка на нее приводит в функцию `0x1400d2740-0x1400d3054`, которую я назвал `audit_main`.

Сначала `audit_main` проходит по глобальному вектору проверок. Для каждой записи он запускает команду через `popen`, читает вывод и ищет строку `OK`. В псевдокоде сравнение выглядит как:

```
if (line.size() == 2 && *(uint16_t *)line.data() == 0x4b4f)
    ++passed;
```

`0x4b4f` в little endian - это ASCII `OK`.

Токен строится только если `passed` равен числу элементов вектора. В успешной ветке код вызывает функцию `0x1400014e0`, выделяет строку на 34 байта и запускает второй LCG:

АУДИТ
REVERSE

05

НАХОДИМ МЕСТО ПЕЧАТИ ТОКЕНА

ПРОДОЛЖЕНИЕ

```
    {
      ((void (__fastcall *)(char *, _QWORD))unk_1400C94C0)(
        a1: (char *)off_1400E0080 + *(_QWORD *)(*(_QWORD *)off_1400E0080 - 24LL),
        a2: *(_DWORD *)((char *)off_1400E0080 + *(_QWORD *)(*(_QWORD *)off_1400E0080 - 24LL) + 32) | 1u);
    }
    ((void (__fastcall *)(void *, void *, __int64))unk_1400CBA80)(a1: off_1400E0080, a2: &unk_1400D922F, a3: 2);
LABEL_40:
    if ( v41 != v43 )
      ((void (__fastcall *)(_QWORD *, __int64))unk_1400D1CC0)(a1: v41, a2: v43[0] + 1LL);
    v1 += 4;
  }
  while ( v1 != (_QWORD *)v39 );
LABEL_43:
  if ( v40 == (unsigned int)((qword_140108038 - qword_140108030) >> 5) )
  {
    v30 = derive_token_seed();
    v0 = (void **)&v49;
    v50 = 0;
    v49 = v51;
    v31 = v30;
    LOBYTE(v51[0]) = 0;
    ((void (__fastcall *)(_BYTE **, __int64, _QWORD))unk_1400BC950)(a1: &v49, a2: 34, a3: 0);
    for ( j = 0; j != 34; ++j )
    {
      v31 = 0x5851F42D4C957F2DLL * v31 + 0x14057B7EF767814FLL;
      v49[j] = byte_1400DA160[j] ^ HIBYTE(v31);
    }
    ((void (__fastcall *)(void *, void *, __int64, __int64))unk_1400CF270)(
      a1: off_1400E0080,
      a2: &unk_1400D9238,
      a3: 0x14057B7EF767814FLL,
      a4: 0x5851F42D4C957F2DLL);
    v33 = ((__int64 (__fastcall *)(void *, const char *))unk_1400CF270)(a1: off_1400E0080, a2: "ACCESS TOKEN: ");
    v34 = ((__int64 (__fastcall *)(__int64, _BYTE *, unsigned __int64))unk_1400CBA80)(a1: v33, a2: v49, a3: v50);
    ((void (__fastcall *)(__int64, const char *))unk_1400CF270)(a1: v34, a2: "\n");
    if ( v50 != 0 )
      memset(v49, 0, v50);
    *v49 = 0;
    v50 = 0;
    ((void (__fastcall *)(_BYTE **))unk_1400BD1E0)(a1: &v49);
    if ( v49 != (_BYTE *)v51 )
      ((void (__fastcall *)(_BYTE *, __int64))unk_1400D1CC0)(a1: v49, a2: v51[0] + 1LL);
  }
  else
  {
    v0 = (void **)&v49;
    ((void (__fastcall *)(void *, void *, __int64))unk_1400CBA80)(a1: off_1400E0080, a2: &unk_1400D9288, a3: 84);
  }
  v49 = v51;
UNKNOWN audit_main:296 (1400D2E44)
```

В этой точке задача разделяется на две части:

АУДИТ
REVERSE

05

НАХОДИМ МЕСТО ПЕЧАТИ ТОКЕНА

ПРОДОЛЖЕНИЕ

- восстановить seed из `derive_token_seed`;
- повторить 34 итерации финального LCG.

АУДИТ
REVERSE

06

ВОССТАНАВЛИВАЕМ SEED

Функция `0x1400014e0` короткая. Я назвал ее `derive_token_seed`.

```
__int64 derive_token_seed()
{
  __int64 v0; // r10
  __int64 result; // rax
  unsigned __int8 *v2; // rdx
  __int64 v3; // r8
  unsigned __int8 *v4; // rdx
  __int64 v5; // r8

  v0 = qword_140108030;
  for ( result = 0xCBF29CE484222325uLL; v0 != qword_140108038; v0 += 32 )
  {
    v2 = *(unsigned __int8 **)(v0 + 16);
    v3 = *v2;
    result = 0x100000001B3LL * (v3 ^ result);
    if ( (_BYTE)v3 != 0 )
    {
      v4 = v2 + 1;
      do
      {
        v5 = *v4++;
        result = 0x100000001B3LL * (v5 ^ result);
      }
      while ( (_BYTE)v5 != 0 );
    }
  }
  return result;
}
```

UNKNOWN derive_token_seed:1 (1400014E0)

06

ВОССТАНАВЛИВАЕМ SEED

ПРОДОЛЖЕНИЕ

Константы сразу выдают FNV-1a 64-bit:

```
offset basis = 0xcbf29ce484222325
prime        = 0x100000001b3
```

Глобальный вектор лежит между `qword_140108030` и `qword_140108038`. Размер одной записи равен `0x20`, а поле по смещению `+0x10` содержит указатель на PowerShell-команду.

Статическая таблица по RVA `0xda060` содержит восемь записей:

N	ID	НАЗНАЧЕНИЕ КОМАНДЫ
0	`av_realtime`	Проверка realtime-защиты Defender
1	`disk_bitlocker`	Проверка BitLocker для `C:`
2	`firewall_all`	Проверка всех профилей firewall
3	`audit_proc`	Проверка аудита Process Creation
4	`dlp_agent`	Проверка службы `CorpDLPAgent`
5	`domain_gpo`	Проверка примененных GPO
6	`szi_state`	Противоречивая проверка `SecureGuard`
7	`audit_pair`	Поиск пары событий аудита 4719

06

ВОССТАНАВЛИВАЕМ SEED

ПРОДОЛЖЕНИЕ

Функция хеширует команды именно в этом порядке. В хеш каждой строки входит и завершающий нулевой байт. Эквивалентный код:

```
state = 0xCBF29CE484222325
prime = 0x100000001B3

for command in commands:
    for value in command + b"\0":
        state = ((state ^ value) * prime) & 0xFFFFFFFFFFFFFFFF
```

Для таблицы из бинаря получается:

```
seed = 0x24c281c32b2bbbe0
```

АУДИТ
REVERSE

07

РАСШИФРОВЫВАЕМ ТОКЕН

Зашифрованный массив начинается по адресу `0x1400da160` и имеет длину 34 байта. Финальный генератор использует другие 64-битные константы:

```
A = 0x5851f42d4c957f2d
B = 0x14057b7ef767814f
```

На каждой итерации:

```
state = (A * state + B) & 0xFFFFFFFFFFFFFFFF
plain[i] = encrypted[i] ^ (state >> 56)
```

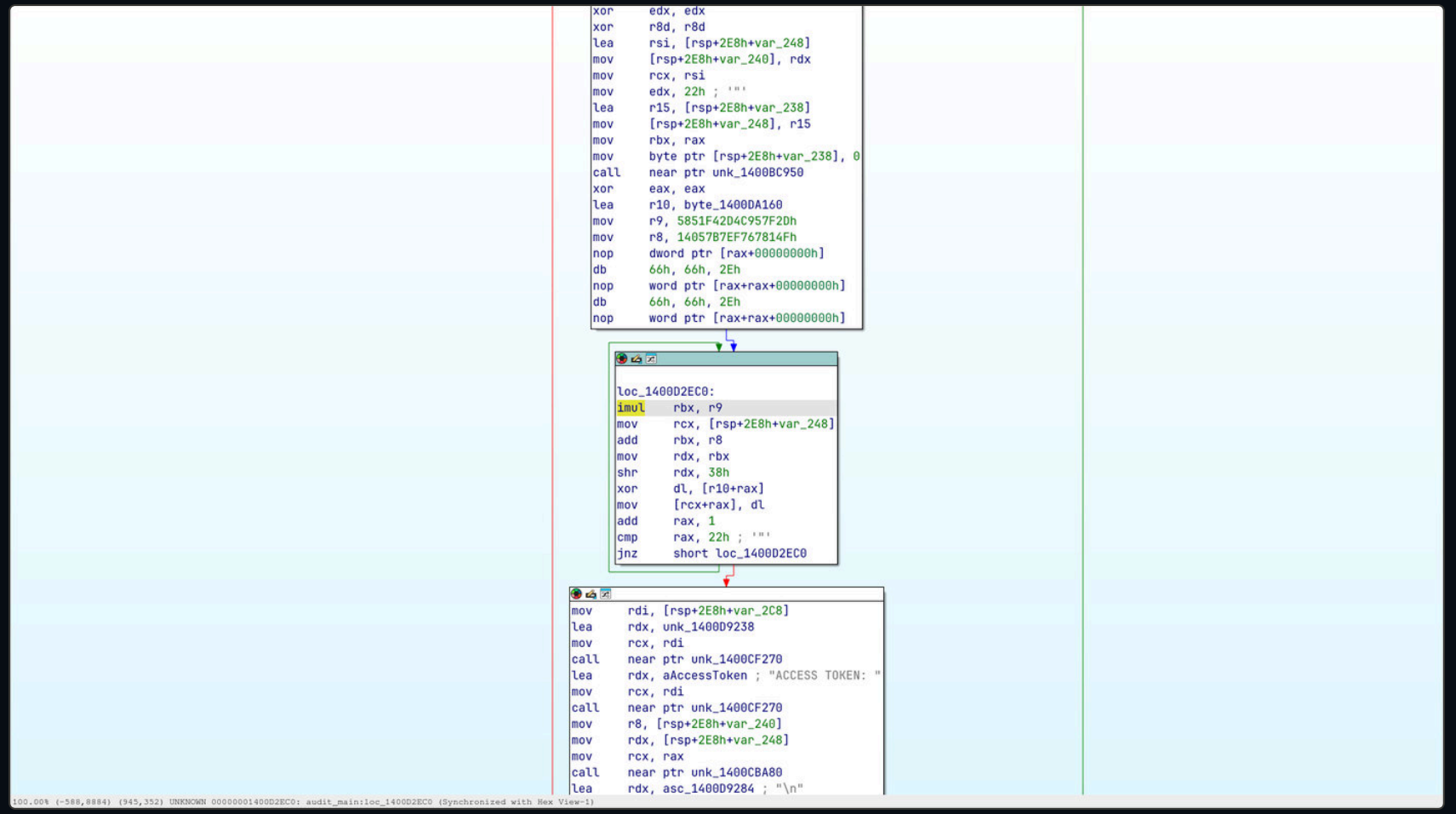
Здесь есть маленькая ловушка. Hex-Rays показывает `HIBYTE(v31)`, что без проверки легко принять за байт `8-15`. В дизассемблере явно стоит `shr rdx, 38h`, поэтому нужен именно старший байт 64-битного состояния.

АУДИТ
REVERSE

07

РАСШИФРОВЫВАЕМ ТОКЕН

ПРОДОЛЖЕНИЕ



АУДИТ
REVERSE

07

РАСШИФРОВЫВАЕМ ТОКЕН

ПРОДОЛЖЕНИЕ

Полная логика вынесена в solve.py. Скрипт читает указатели на команды прямо из восстановленного образа, вычисляет FNV-1a и расшифровывает массив токена.

АУДИТ
REVERSE

08

ПОЛУЧАЕМ ФЛАГ

Финальный запуск:

```
python3 unpack.py
python3 solve.py
```

Результат:

```
flag{h1st0ry_s4v3d_c0d3_r3wr1tt3n}
```

В итоге чекер не пришлось запускать и не пришлось воспроизводить невозможную конфигурацию рабочей станции. Достаточно было снять первый слой упаковки, восстановить таблицу проверок и повторить два небольших алгоритма из успешной ветки.

АУДИТ
REVERSE

ФЛАГ

```
flag{h1st0ry_s4v3d_c0d3_r3wr1tt3n}
```

Задание «Аудит» решено